

Tablice w Java

Zbigniew Marszalek

Tablice

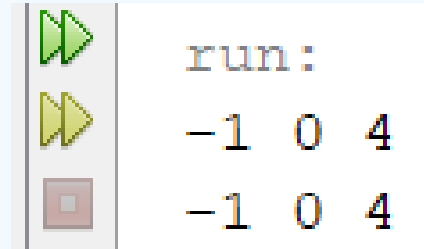
Tablice są indeksowanym ciągiem elementów tego samego typu. Referencja do tablicy jednowymiarowej jest deklarowana za pomocą składni:

```
<T> nazwa_tablicy[]
```

Poniższy program ilustruje sposób deklaracji uchwytu do tablicy liczb zmiennoprzecinkowych, łańcuchów znakowych i inicjalizacji tablicy liczb całkowitych.

```
    double [] a;  
// Deklaracja referencji do tablicy liczb  
//zmiennoprzecinkowych.  
    String b[];  
// Deklaracja uchwytu do tablicy łańcuchów znakowych.  
    int c[] = {-1, 0, 4};  
// Deklaracja tablicy liczb całkowitych z inicjalizacją  
//tablicy.  
    for(int i=0; i < c.length; i++)  
        System.out.printf("%d ",c[i]);  
// Wydruk liczb zapisanych w tablicy za pomocą pętli for  
    System.out.println();  
// Przejsćie do nowej linii.  
    for(int x: c)  
        System.out.printf("%d ",x);
```

Konsola w wydrukiem liczb zapisanych w tablicy



```
run:
-1 0 4
-1 0 4
```

Wydruk liczb zapisanych w tablicy został wykonany za pomocą pętli for i instrukcji for-each. Warto podkreślić, że obiekt tablica przechowuje ilość zapisanych w nim elementów. Użytkownik może tylko odczytać ilość przechowywanych elementów w tablicy za pomocą odpowiedniej właściwości, ale nie może dokonywać zmian w wielkości przechowywanych elementów przez obiekt. Może natomiast zmieniać zapisane wartości w tablicach zgodnie z zaprojektowanymi algorytmami.

Do konstrukcji tablicy używamy słowa kluczowego new. Poniżej podajemy przykłady konstrukcji tablic.

```
String [] t = {"Kowalski", "Nowak", "Jąąąąąąąąąą"};  
for(String s: t)  
    System.out.println(s);  
// Wydruk zawartość tablicy.  
double [] b = new double[4];  
// Konstrukcja tablicy typu double zawierającej  
// cztery elementy.  
for(int i=0; i < b.length; i++)  
    b[i]= 0.1*i;  
// Nadanie wartości początkowych elementom tablicy.  
for(double x: b)  
    System.out.println(x);  
// Wydruk zawartości tablicy
```


Dostęp do elementów tablic

Spróbujemy zaimplementować metodę w klasie algorytm wyszukującą indeks najmniejszego elementu w tablicy liczb zmiennoprzecinkowych.

```
package c4b;
public class Algorytm {
    public int min(double a[]) {
        // Metoda wyszukiwania najmniejszego elementu.
        int t=0;
        // Przyjęcie, że elementem najmniejszym
        //jest pod indexm 0.
        double min = a[0];
        // Zapamiętanie wartości elementu.
```

```
    for(int i=1;i<a.length;i++)  
// Przeszukanie wszystkich elementów tablicy.  
    if(a[i]<min) {  
// Sprawdzenie, czy kolejny element jest mniejszy od  
// uprzednio znalezionego.  
        t=i;  
// Zapamiętanie indeksu elementu.  
        min=a[i]; }  
// Zapamiętanie wartości elementu.  
    return t;  
// Zwrócenie indeksu elementu najmniejszego.  
}  
}
```

```
package c4b;
import java.util.*;
public class C4b {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Podaj ilość wlemntów: ");
        int n = in.nextInt();
        // Przeczytanie ilości elementów do przeczytania.
        double[] a = new double[n];
        // Konstrukcja tablicy.
        for(int i=0; i < a.length; i++){
            // Pobranie kolejnych elementów do tablicy.
            System.out.printf("Podaj element - %d: ", i);
            a[i]=in.nextDouble();
        }
    }
}
```

```
    Algorytm at = new Algorytm();  
    // Umieszczenie klasy na zarządzanej stercie.  
    int ndx = at.min(a);  
    // Wywołanie metody znajdujacej indeks.  
    System.out.printf  
        ("Element najmniejszy: a[%d]=%g%n",  
         ndx,a[ndx]);  
    // Wydruk najmniejszego elementu.  
}  
}
```



run:

Podaj ilość wlemntów: 4

Podaj element - 0: 4

Podaj element - 1: -30

Podaj element - 2: 6

Podaj element - 3: -5

Element najmniejszy: a[1]=-30,0000

Program najpierw wczytuje ilość elementów do pobrania z konsoli. Następnie konstruuje tablicę liczb zmiennoprzecinkowych i wczytuje do niej liczby przeczytane z konsoli. Liczby nie są sprawdzane pod względem poprawności konwersji przeczytanych łańcuchów znakowych. Zakładamy, że użytkownik poprawnie wprowadzi liczby. W dalszej części książki postaramy się rozwiązać problem kontroli poprawności wprowadzanych danych. Ze względu na polonizację pakietu Visual Studio przecinek oddziela część ułamkową od całkowitej. Próba wprowadzenia kropki spowoduje błąd i przerwanie działania programu. W dalszej kolejności działania programu zostaje wywołana metoda wyszukiwania indeksu najmniejszej liczby i wydruk najmniejszego elementu w ciągu liczbowym. Na konsoli użytkownika wygląda to następująco.

Przeciążenie metody wyszukiwania najmniejszego elementu w tablicy

Przypuśćmy, że potrzebujemy metodę w klasie Algorytm wyszukiującą najmniejszy element w tablicy w przedziale tzn. od zadanego dolnego indeksu w tablicy do drugiego indeksu górnego

$$0 \leq imin \leq i < imax \leq a.Length$$

Metoda ta będzie miała taką samą nawę, lecz inną ilość parametrów wejściowych. Oprócz nazwy tablicy metoda będzie posiadała dwa parametry wskazujące na indeksy tablicy, definiujące zakres przeszukania.

Zdarzenie polegające na kontekstowym wywołaniu metody w zależności od środowiska jej wywołania nosi nazwę polimorfizmu.

```
package c4c;
public class Algorytm {
    // Klasa Algorytm.
    public int min(double[] a) {
        // Metoda wyszukiwania najmniejszego elementu
        // w tablicy.
        int t=0;
        double min = a[0];
        for(int i=1;i<a.length;i++)
            // Przeszukanie wszystkich elementów tablicy.
            if(a[i]<min) {
                t=i;
                min=a[i];
            }
        return t;
    }
}
```

```
public int min(double[] a,int imin,int imax){  
    // Metoda przeciążona wyszukiwania najmniejszego  
    // elementu w tablicy pomiędzy zadanymi indeksami  
    // imin <= i < imax.  
    if(imin < 0 || imin > a.length) imin = 0;  
    //Sprawdzenie czy indeks imin jest poprawnie zadany.  
    if (imax > a.length || imax < 0) imax = a.length;  
    // Sprawdzenie czy indeks imax jest poprawnie zadany  
    int t = imin;  
    //Przyjęcie, elementem najmniejszym - indeks imin.  
    double min = a[imin];  
    // Zapamiętanie wartości elementu zerowego.
```

```
    for (int i = imin + 1; i < imax; i++)  
// Przeszukanie wszystkich elementów tablicy.  
    if (a[i] < min) {  
// Sprawdzenie, czy kolejny element jest  
// mniejszy od uprzednio znalezionego.  
        t = i;  
// Zapamiętanie indeksu elementu.  
        min = a[i]; }  
// Zapamiętanie wartości elementu.  
    return t;  
// Zwrócenie indeksu elementu najmniejszego.  
}  
}
```

```
import java.util.*;
public class C4c {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Podaj ilość wlemntów: ");
        int n = in.nextInt();
        // Przeczytanie ilości elementów do przeczytania.
        double[] a = new double[n];
        // Konstrukcja tablicy.
        for(int i=0; i < a.length; i++){
            // Pobranie kolejnych elementów do tablicy.
            System.out.printf("Podaj element - %d: ", i);
            a[i]=in.nextDouble();
        }
    }
}
```

```
System.out.print("Podaj indeks dolnu: ");
    int imin = in.nextInt();
// Wczytanie indeksu, od którego nastąpi poszukiwanie.
    System.out.print("Podaj indeks górnego: ");
    int imax = in.nextInt();
// Wczytanie indeksu, do którego nastąpi przeszukanie.
// Indeks tego elementu, nie jest barany pod uwagę.
    Algorytm at = new Algorytm();
// Umieszczenie klasy na zarządzanej stercie.
    int ndx = at.min(a, imin, imax);
    System.out.printf
        ("Element najmniejszy: a[%d]=%g%n",
         ndx, a[ndx]);
}
}
```

Działanie programu



```
run:
Podaj ilość wlemntów: 8
Podaj element - 0: -90
Podaj element - 1: 2
Podaj element - 2: -3
Podaj element - 3: 3
Podaj element - 4: -1
Podaj element - 5: 9
Podaj element - 6: 3
Podaj element - 7: 10
Podaj indeks dolnu: 1
Podaj indeks górnego: 5
Element najmniejszy: a[2]=-3,00000
```

Sortowanie zawartości tablic

Elementy umieszczone w tablicy możemy uporządkować od najmniejszego do największego za pomocą metody *sort()* z klasy *Arrays*.

```
int[] a = { -3, -9, 2, 3, 4, 0 };  
System.out.println  
    ("Wydruk wartości z tablicy");  
for(int i = 0; i < a.length; i++)  
    System.out.print(a[i] + " ");  
System.out.println();  
Arrays.sort(a);  
for (int x: a)  
    System.out.print(x + " ");  
System.out.println();
```

run:

```
Wydruk wartości z tablicy  
-3 -9 2 3 4 0  
-9 -3 0 2 3 4
```

Sortowanie liczb *Integer* w odwrotnej kolejności

```
package c4e;  
import java.util.Collections;  
import java.util.Arrays;  
public class C4e {  
    static void wydruk(Integer a[]) {  
        // Funkcja drukujaca tablice.  
        for(int i=0;i<a.length;i++)  
            System.out.printf("%d ",a[i]);  
        System.out.println();  
    }  
}
```

```
public static void main(String[] args) {  
    Integer[] a = {-3,-9,2,3,4,0};  
    // Należy zauważyć, że używamy Integer[] zamiast  
    // int[] jako kolekcji. Collections.reverseOrder  
    // nie działa z prymitywnymi typami danych.  
    System.out.println  
        ("Wydruk wartości z tablicy");  
    wydruk(a);  
    // Wydruk zawartości tablicy.  
    Arrays.sort(a, Collections.reverseOrder());  
    // Sortowanie tablicy za pomocą nowego porwania.  
    System.out.println  
        ("Wydruk elementów posortowanych");  
    wydruk(a);  
}}
```

Sortowanie elementów w tablicy poprzez metodę *reverseOrder()*



```
run:
Wydruk wartości z tablicy
-3 -9 2 3 4 0
Wydruk elementów posortowanych
4 3 2 0 -3 -9
```

Sortowanie liczb *Number* w odwrotnej kolejności

```
package c4e1;
import java.util.Collections;
import java.util.Arrays;
public class C4e1 {
    static void wydruk(Number a[]) {
// Funkcja drukujaca tablice.
        for(int i=0;i<a.length;i++)
            System.out.printf("%g ",a[i]);
        System.out.println();
    }
}
```

```
public static void main(String[] args) {  
    Number[] a = {-3.0, -9.1, 2.8, 3.2, 4.1, 0.0};  
    // Należy zauważyć, że używamy Integer[] zamiast  
    // int[] jako kolekcji. Collections.reverseOrder  
    // nie działa z prymitywnymi typami danych.  
    System.out.println  
        ("Wydruk wartości z tablicy");  
    wydruk(a);  
    // Wydruk zawartości tablicy.  
    Arrays.sort(a, Collections.reverseOrder());  
    // Sortowanie tablicy za pomocą nowego porwania.  
    System.out.println  
        ("Wydruk elementów posortowanych");  
    wydruk(a);  
    // Wydruk zawartości tablicy.  
}}
```

Sortowanie elementów w tablicy poprzez metodę *reverseOrder()*



run:



Wydruk wartości z tablicy



-3,00000 -9,10000 2,80000 3,20000 4,10000 0,00000



Wydruk elementów posortowanych

4,10000 3,20000 2,80000 0,00000 -3,00000 -9,10000

Metoda *sort()* została zaprojektowana z myślą o sortowaniu łańcuchów znakowych oraz wartości liczbowych. Istnieje możliwość wykorzystania tej metody do sortowania obiektów po zaimplementowaniu własnej metody w klasie obiektów będących przedmiotem takiego sortowania. Metoda *compareToIgnoreCase()* porównuje dwa łańcuchy ignorując wielkość znaków i jej składnia jest następująca

int compareToIgnoreCase(String a1, String a2).

Zwracana wartość	Warunek
Liczba mniejsza od zera	Obiekt pierwszy jest mniejszy od obiektu drugiego
Zero	Obiekty są równe
Liczba większa od zera	Obiekt pierwszy jest większy od obiektu drugiego

```
private static int compareIgnoreCase
    (String a1,String a2)
{
    Locale locale =new Locale("pl","PL");
//    Utworzenie obiektu lokalnych ustawień
    Collator col =
        Collator.getInstance(locale);
//Utworzenie komparatora instancji polskiej.
    col.setStrength(Collator.PRIMARY);
//    Wpisanie sposobu porównywania znaków.
    return col.compare
        (a1.toUpperCase(locale),
         a2.toUpperCase(locale));
//    Zwrócenie wyniku porównania.
}
```

Sortowanie obiektów klasy ***Pracownik*** za pomocą metody *compareTo()*

```
package c4f;  
import java.text.*;  
import java.util.*;  
class Pracownik implements Comparable {  
    private int id;  
    private String nazwisko, telefon;  
    // Pola klasy.  
    public Pracownik() { }  
    // Konstruktor domyślny.
```

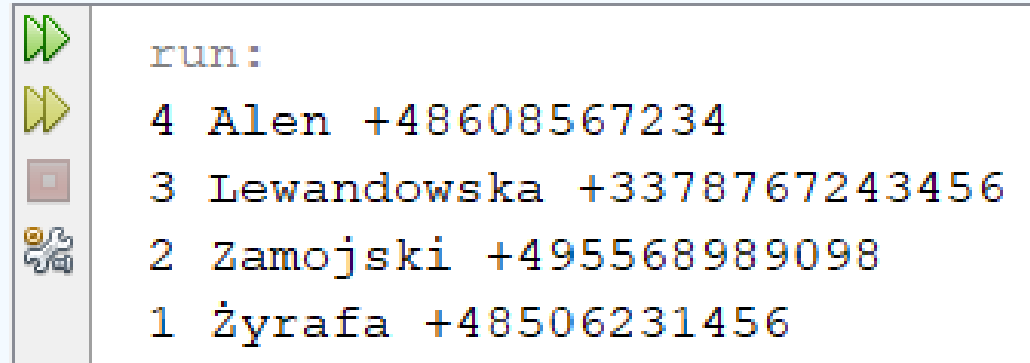
```
public Pracownik(int id, String nazwisko,  
                  String telefon) {  
    // Przeciążenie konstruktora.  
    this.id = id;  
    this.nazwisko = nazwisko;  
    this.telefon = telefon;  
}  
  
public String toString() {  
    // Przeciążenie metody ToString().  
    return String.format  
        ("%d %s %s", id, nazwisko,  
                    telefon);  
}
```

```
private static int compareIgnoreCase
    (String a1,String a2)
{
    Locale locale =new Locale("pl","PL");
//    Utworzenie obiektu lokalnych ustawień
    Collator col =
        Collator.getInstance(locale);
//Utworzenie komparatora instancji polskiej.
    col.setStrength(Collator.PRIMARY);
//    Wpisanie sposobu porównywania znaków.
    return col.compare
        (a1.toUpperCase(locale),
         a2.toUpperCase(locale));
//    Zwrócenie wyniku porównania.
}
```

```
    public int compareTo(Object obj) {  
        if (this==obj) return 0;  
        if (this == null) return -1;  
        if (this.getClass() != obj.getClass()  
            ) return 1;  
        Pracownik e = (Pracownik) obj ;  
        int t = compareIgnoreCase  
            (this.nazwisko,e.nazwisko) ;  
        return t;  
    }  
}  
  
public class C4f {  
    public static void main(String[] args) {
```

```
Pracownik[] a = new Pracownik[4];  
// Konstrukcja tablicy na zarządzanym stosie.  
a[0] = new Pracownik  
    (1, "Żyrafa", "+48506231456");  
a[1] = new Pracownik  
    (2, "Zamojski", "+495568989098");  
a[2] = new Pracownik  
    (3, "Lewandowska", "+3378767243456");  
a[3] = new Pracownik  
    (4, "Alen", "+48608567234");  
// Wpisanie rekordów.  
    Arrays.sort(a);  
    for (Pracownik c: a)  
        System.out.println(c);  
// Wydruk zawartości tablicy.  
}}
```

Wynik sortowania



```
run:
4 Alen +48608567234
3 Lewandowska +3378767243456
2 Zamojski +495568989098
1 Żyrafa +48506231456
```

Napisanie metody sortującej według jednego pola nie jest trudne. Wystarczy napisać metodę *compareToIgnoreCase()* porównującą prawidłowo polskie znaki i w metodzie *compareTo()* wykorzystać zaprojektowaną metodę tak aby wykonywała porównanie wg. pierwszego porównywanego pola.

Implementujemy algorytm porównywania rekordów zawierających numer pracownika, nazwisko i numer telefonu wg. nazwiska od A do Ż (ang. *ascending*).

Sortowanie rekordów wg. dwóch pól

Napisanie metody sortującej według kilku pól nie jest trudne. Wystarczy odpowiednio zmodyfikować metodę *compareTo()*, tak aby wykonywała porównanie najpierw wg. pierwszego porównywanego pola i przechodziła do porównania drugiego pola przy równości elementów w pierwszym polu. Zastosujemy algorytm porównywania dwóch pól do posortowania rekordów zawierających numer pracownika, nazwisko i zajmowane stanowisko wg. zajmowanego stanowiska i nazwiska z odwróconą kolejnością od Ż do A (ang. *descending*).

```
package c4g;
import java.text.*;
import java.util.*;
class Pracownik implements Comparable {
    private int id;
    private String nazwisko, stanowisko;
// Pola klasy.
    public Pracownik() { }
// Konstruktor domyślny.
    public Pracownik(int id, String nazwisko,
        String stanowisko){
// Przeciążenie konstruktora.
        this.id = id;
        this.nazwisko = nazwisko;
        this.stanowisko = stanowisko;
    }
}
```

```
public String toString() {  
    return String.format  
        ("%3d %-12s %-10s", id, nazwisko,  
                                stanowisko);  
}  
  
private static int compareIgnoreCase  
    (String a1, String a2) {  
    Locale locale = new Locale("pl", "PL");  
    Collator col =  
        Collator.getInstance(locale);  
    col.setStrength(Collator.PRIMARY);  
    return col.compare  
        (a1.toUpperCase(locale),  
         a2.toUpperCase(locale));  
}
```

```
public int compareTo(Object obj) {  
    if (this==obj) return 0;  
    if (this == null) return -1;  
    if (this.getClass() != obj.getClass()  
        ) return 1;  
    Pracownik e = (Pracownik) obj ;  
    int t = compareIgnoreCase  
        (this.stanowisko,e.stanowisko) ;  
    // Porównanie wg. pierwszego pola.  
    // Alfabetycznie od Ż do A (descending)  
    if(t!=0) return t;  
    t = compareIgnoreCase  
        (e.nazwisko,this.nazwisko) ;  
    return t;  
}  
}
```

```
public class C4g {  
    public static void main(String[] args) {  
        Pracownik[] a = new Pracownik[6];  
        // Konstrukcja tablicy na zarządzanym stosie.  
        a[0] = new Pracownik  
            (1, "Żyrafa", "Analityk");  
        a[1] = new Pracownik  
            (2, "Zamojski", "Sprzedawca");  
        a[2] = new Pracownik  
            (3, "Lewandowska", "Analityk");  
        a[3] = new Pracownik  
            (4, "Alen", "Sprzedawca");  
        a[4] = new Pracownik  
            (4, "Nowak", "Sprzedawca");  
        a[5] = new Pracownik  
            (4, "Woźnica", "Sprzedawca");  
        // Wpisanie rekordów.
```

```
Arrays.sort(a);  
for (Pracownik c: a)  
    System.out.println(c);  
// Wydruk zawartości tablicy.  
}  
}
```



run:



1	Żyrafa	Analityk
3	Lewandowska	Analityk
2	Zamojski	Sprzedawca
4	Woźnica	Sprzedawca
4	Nowak	Sprzedawca
4	Alen	Sprzedawca

Tablice wielowymiarowe

Istnieją dwa sposoby deklaracji tablic wielowymiarowych. Pierwszy sposób umożliwia deklarację tablicy za pomocą pojedynczego uchwytu. Składania deklaracji uchwytu jest następująca.

```
Type   nazwa_tablicy [][]
```

Program ilustrujący deklaracje tablic dwuwymiarowych wygląda następująco.

Mnożenie macierzy przez wektor

Tak jak wspomnieliśmy w poprzednim rozdziale metoda może zwracać utworzony obiekt. Przedstawmy obecnie sposób konstrukcji wektora powstającego w wyniku mnożenia macierzy przez wektor. Spróbujmy teraz napisać metodę mnożenia w klasie referencyjnej Algorytm wykonującą konstrukcję wektora powstającego z pomnożenia macierzy $A \in R^{n \times n}$ przez wektor $x \in R^n$.

$$y = Ax, \quad \text{gdzie } A \in R^{n \times n} \quad x, y \in R^n$$

$$y_i = \sum_{j=0}^{n-1} a_{ij} x_j \quad \text{dla } i = 0, \dots, n-1$$

```
package c4h;
public class Algorytm {
    public double [] mnozenie(double [][] a, double [] x)
// Deklaracja metody
    { int n= x.length;
// Określenie wymiaru zadania.
        double [] y = new double [n];
// Konstrukcja wektora.
        for(int i=0; i < n; i++){
// Rozpisanie algorytmu mnożenia macierz.
            double s=0.0;
            for(int j=0; j < n; j++)
                s+=a[i][j]*x[j];
            y[i]=s;
        }
        return y;
    }
}
```

```
package c4h;
import java.util.*;
public class C4h {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Podaj wymiar zadania: ");
        int n = 0;
        try{
            n= in.nextInt();
        }
        catch (Exception e) {
            System.out.println("Błąd konwersji liczby");
            return; }
        // Przeczytanie wymiaru zadania z sprawdzeniem na
        // poprawność konwersji.
```

```
    if (n < 1) {  
        System.out.println("Wpisz liczbę dodatnią");  
        return;  
    }  
    double a[][] = new double[n][n];  
    // Deklaracja macierzy.  
    double[] x = new double[n];  
    // Deklaracja wektora.  
    try {  
        for (int i = 0; i < n; i++) {  
            // Przeczytanie elementów macierzy i wektora.  
            for (int j = 0; j < n; j++) {  
                System.out.printf  
                    ("Podaż a[%d][%d]: ", i, j);  
                a[i][j] = in.nextDouble();  
            }  
        }  
    }  
}
```

```
        System.out.printf("Podaj x[%d]: ", i);  
        x[i] = in.nextDouble();  
    }  
}  
catch (Exception e) {  
    System.out.println  
        ("Błąd konwersji liczby");  
return; }  
Algorytm at = new Algorytm();  
// Umieszczenie metody na zarządzanej stercie.  
    double[] y = at.mnozenie(a, x);  
// Zwrócenie przez metodę obliczonego wektora.
```

```
    for ( int i = 0; i < n; i++) {  
// Wydruk obliczeń.  
        if (i != n/2)  
            System.out.printf("|%5.1f| |", y[i]);  
        else System.out.printf("|%5.1f|=|", y[i]);  
        for ( int j = 0; j<n; j++)  
            System.out.printf("%5.1f", a[i][j]);  
        System.out.printf("||%5.1f|\n", x[i]);  
    }  
}  
}
```

Wydruk mnożenia macierzy przez wektor

```
run:
Podaj wymiar zadania: 3
Podaj a[0][0]: 1
Podaj a[0][1]: 1
Podaj a[0][2]: 1
Podaj x[0]: 1
Podaj a[1][0]: 0,5
Podaj a[1][1]: 1
Podaj a[1][2]: -0,5
Podaj x[1]: 1
Podaj a[2][0]: -1
Podaj a[2][1]: -1
Podaj a[2][2]: -1
Podaj x[2]: 1
| 3,0| | 1,0 1,0 1,0|| 1,0|
| 1,0|=| 0,5 1,0 -0,5|| 1,0|
| -3,0| | -1,0 -1,0 -1,0|| 1,0|
```

```
run:
Podaj wymiar zadania: 3
Podaj a[0][0]: 1.2
Błąd konwersji liczby
```

Typy dynamiczne

Tablice mają ustalony stały wymiar przy inicjalizacji. Jeżeli w czasie pracy musimy zmieniać wielkość tablicy należy zastosować klasę *ArrayList*. Klasa *ArrayList* jest umieszczona w przestrzeni nazw `java.util.colections`, którą musimy dołączyć do naszego projektu. W poniższym przykładzie pokazano jak posługiwać się zarządzaną tablicą

```
        ArrayList a = new ArrayList();  
// Deklaracja uchwytu i inicjalizacja ArrayList.  
        a.add("Element_1");  
        a.add(2);  
        a.add(1.2e-3);  
// Dopisanie elementów
```

```
        System.out.println
            ("ArrayList po wpisaniu elementów");
        for(int i=0;i< a.size() ;i++)
// Wydruk zawartości ArrayList.
        System.out.println(a.get(i)+" ");
        System.out.println();
        a.add(1,"Element_2");
// Dopisanie elementu na pozycję pierwszą.
        a.remove(1.2e-3);
// Usunięcie elementu o wartości 1.2e-3.
        System.out.println
            ("Po dopisaniu i usunięciu elementu");
        for(int i=0; i < a.size() ; i++)
// Wydruk zawartości ArrayList.
        System.out.println(a.get(i)+" ");
        System.out.println();
```

Wydruk elementów z klasy ArrayList



```
ArrayList po wpisaniu elementów
```



```
Element_1
```



```
2
```



```
0.0012
```



```
Po dopisaniu i usunięciu elementu  
Element_1  
Element_2  
2
```

ArrayList

ArrayLista jest obiektem podobnym do tablicy, ale oprócz właściwości przechowującej ilość elementów w obiekcie posiada metody umożliwiające dodawanie elementów do listy oraz metody umożliwiające usuwanie elementy z listy.

UWAGA. Do programu należy zaimportować przestrzeń nazw:

```
import java.util.*;
```

Meoda	Opis
boolean add(E element)	Dodaje element na koniec listy
Void add(int index, E element)	Metoda wstawia element pod zadany indeks i przesuwa wszystkie elementy listy.
E get(int index)	Metoda zwraca element z pod pozycji o zadany indeksie
E remove(int index)	Metoda usuwa element z pod danego indeksu listy
E set(int index, E element)	Metoda podstawia zadany element pod indeks listy
int size()	Metoda zwraca liczbę elementów w liście

```
import java.text.*;
import java.util.*;
class Pracownik implements Comparable {
    private int id;
    private String nazwisko, telefon;
// Pola klasy.
    public Pracownik() { }
// Konstruktor domyślny.
    public Pracownik(int id, String nazwisko,
                     String telefon) {
// Przeciążenie konstruktora.
        this.id = id;
        this.nazwisko = nazwisko;
        this.telefon = telefon;
    }
}
```

```
public String toString() {  
    return String.format  
        ("%3d %-12s %-10s", id, nazwisko,  
            telefon);  
}  
  
private static int compareIgnoreCase  
    (String a1,String a2) {  
    Locale locale =new Locale("pl","PL");  
    Collator col =  
        Collator.getInstance(locale);  
    col.setStrength(Collator.PRIMARY);  
    return col.compare  
        (a1.toUpperCase(locale),  
            a2.toUpperCase(locale));  
}
```

```
public int compareTo(Object obj) {  
    if (this==obj) return 0;  
    if (this == null) return -1;  
    if (this.getClass() != obj.getClass()  
        ) return 1;  
    Pracownik e = (Pracownik) obj ;  
    int t = compareIgnoreCase  
        (this.nazwisko,e.nazwisko);  
    return t;  
}  
  
}  
  
public class C4j {  
    public static void main(String[] args) {  
        ArrayList<Pracownik> b = new ArrayList<>();  
        // Konstrukcja listy.
```

```
b.add( new Pracownik
    (1, "Żyrafa", "+48506231456"));
b.add( new Pracownik
    (2, "Zamojski", "+495568989098"));
b.add( new Pracownik
    (3, "Lewandowska", "+3378767243456"));
b.add( new Pracownik
    (4, "Alen", "+48608567234"));
// Wpisanie rekordów.
Collections.sort(b);
for(Pracownik c: b)
    System.out.println(c);
// Wydruk zawartości tablicy.
}
}
```

Wydruk posortowanych elementów



```
run:
    4 Alen +48608567234
    3 Lewandowska +3378767243456
    2 Zamojski +495568989098
    1 Żyrafa +48506231456
```

Gdybyśmy próbowali posortować obiekt klasy *ArrayList* bez napisania własnej metody porównywania obiektów w klasie zostanie zwrócony błąd, ponieważ kompilator nie będzie wiedział jakiego typu obiekty są ze sobą porównywane.

Stosy

Struktura danych w stosie typu ostatni na wejściu, pierwszy na wejściu (ang. *Last In First Out - LIFO*) umożliwia odkładanie elementów na stos i ich pobieranie ze stosu. W języku Java znajdujemy instrukcję umożliwiającą budowę stosu.

Stack a = **new** Stack();

Nowe elementy odkładamy na stos za pomocą metody *push()*. W celu odczytania elementu znajdującego się na stosie bez jego zdejmowania używamy metody *peek()*.

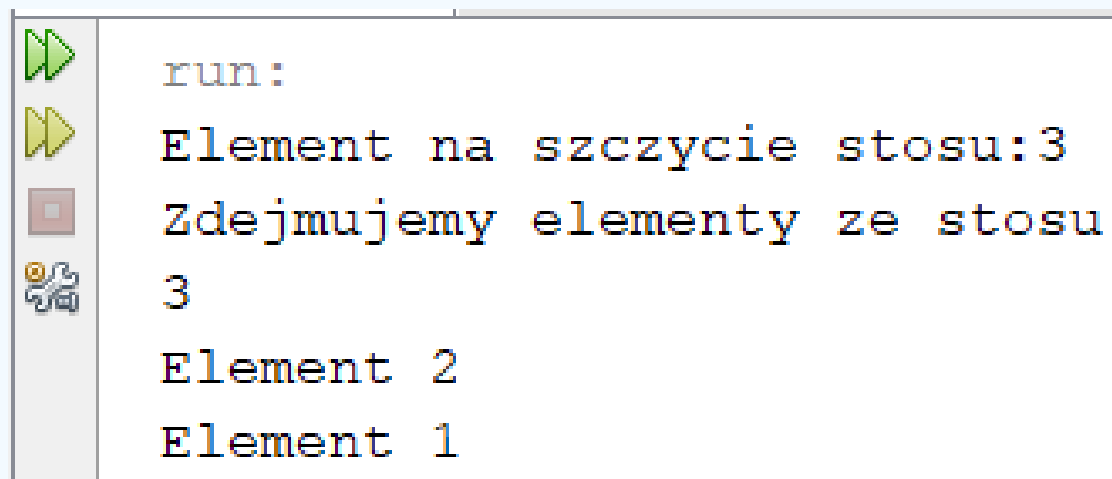
Metoda *pop()* odczytuje i zdejmuje element ze stosu.

Poniższy program ilustruje budowę stosu i wykorzystanie metod do organizacji stosu.

```
Stack a= new Stack();  
// Budowa stosu.  
a.push("Element 1");  
a.push("Element 2");  
a.push(3);  
// Wstawienie elementów na stos.  
System.out.println  
    ("Element na szczycie stosu:" + a.peek());  
//Przeczytanie elementu znajdującego się na stosie.  
System.out.println  
    ("Zdejmujemy elementy ze stosu");  
while( a.size() > 0)  
    System.out.println(a.pop());  
//Zdjęcie i wypisanie wszystkich elemntów ze stosu.
```

Wydruk elementów na stosie

Jeżeli stos jest pusty i będziemy próbowali zdjąć z niego element zostanie ujawniony wyjątek . Dlatego przed próbą zdjęcia elementu ze stosu należy sprawdzić rozmiar stosu za pomocą właściwości *size()* tak jak zostało zrobione to w prezentowanym przykładzie.

A screenshot of a Python IDE's console window. On the left side, there is a vertical toolbar with four icons: a green double arrow (run), a yellow double arrow (debug), a red square (stop), and a magnifying glass (search). The console area on the right displays the following text:

```
run:  
Element na szczycie stosu:3  
Zdejmujemy elementy ze stosu  
3  
Element 2  
Element 1
```

Kolejki

Struktura danych w kolejce typu pierwszy na wejściu, pierwszy na wyjściu (ang. *First In First Out* - *FIFO*) umożliwia dodawanie elementów do kolejki pobieranie ich z kolejki. W języku Java znajdujemy instrukcję umożliwiającą budowę kolejki.

Queue a = **new** LinkedList();

Nowe elementy dodajemy do kolejki za pomocą metody *poll()* albo *remove()*. W celu odczytania elementu znajdującego się na początku kolejki bez jego usuwania używamy metody *peek()*.

Metoda *poll()* i *remove()* odczytuje element i usuwa z kolejki.

Poniższy program ilustruje budowę kolejki i wykorzystanie metod do organizacji kolejki.

```
Queue a=new LinkedList<>();  
// Budowa kolejki.  
a.add("Element 1");  
a.add("Element 2");  
a.add(3);  
// Wstawienie elementów do kolejki.  
System.out.println  
    ("Element na początku kolejki: "+ a.peek());  
// Przeczytanie elementu znajdującego na początku kolejki.  
System.out.println  
    ("Pobieramy i usuwamy kolejne elementy z kolejki");  
while(a.size()>0)  
    System.out.println(a.poll());  
// Pobranie i wypisanie wszystkich elementów z kolejki.
```

Wydruk elementów w kolejce

Podobnie jak w przypadku stosu przy opróżnieniu kolejki metodą *remove()* i próbie odczytu zostanie ujawniony wyjątek. Metoda *poll()* zwraca wartość *null*, jeśli kolejka jest pusta. Do kontroli liczby elementów w kolejce wykorzystano metod *size()*, tak jak zostało zrobione to w prezentowanym przykładzie.



```
run:
```

```
Element na początku kolejki: Element 1
```

```
Pobieramy i usuwamy kolejne elementy z kolejki
```

```
Element 1
```

```
Element 2
```

```
3
```

Lista metody i właściwości

```
ArrayList<String> koty = new ArrayList<>();  
// Utworzenie listy łańcuchów znakowych.  
    koty.add("Lampart");  
// Dodanie do listy pojedynczego łańcucha znaków.  
    koty.add("Puma");  
    koty.add("Lew");  
    koty.add("Jaguar");  
    koty.add("Tygrys");  
    for(String kot: koty)  
    {  
        System.out.println(kot);  
    }  
    System.out.println("Count: " + koty.size());  
// Wypisanie właściwości Count.
```

```
System.out.println();
System.out.println("Indeks(\"Jaguar\") : "+
koty.contains("Jaguar"));
System.out.println("Insert(2, \"Tygrys\")");
koty.add(2, "Tygrys");
//Wstawienie łańcucha do listy w miejsce o
//indeksie 2.
    for(String kot: koty )
        System.out.println(kot);
// Wypisanie zawartości listy.
    System.out.println("koty(3): " + koty.get(3));
// Wypisanie trzeciego elementu z listy.
    System.out.println("Remove(\"Tygrys\")");
    koty.remove("Tygrys");
// Usunięcie łańcucha "Tygrys".
```

```
System.out.println();  
for(String kot: koty )  
    System.out.println(kot);  
// Wypisanie zawartości listy.  
    koty.trimToSize();  
// Usunięcie nadmiarowych elementów listy.  
    System.out.println("Count: " + koty.size());  
// Wydruk właściwości Count.  
    koty.clear();  
// Usunięcie wszystkich elementów listy.  
    System.out.println("Count: " + koty.size());  
// Wydruk właściwości Count.
```

Wydruk z programu



```
run:
Lampart
Puma
Lew
Jaguar
Tygrys
Count: 5

Indeks("Jaguar"): true
Insert(2, "Tygrys")
Lampart
Puma
Tygrys
Lew
Jaguar
Tygrys
koty(3): Lew
Remove("Tygrys")
```

```
Lampart
Puma
Lew
Jaguar
Tygrys
Count: 5
Count: 0
```